

React App

Last updated by | Ezhova Olya | Oct 31, 2025 at 4:53 PM CDT

Contents

- [1. Introduction](#)
- [2. Technical Stack](#)
- [3. Project Structure Overview](#)
- [4. Entry Points: App.tsx, main.tsx & loader.tsx](#)
 - [App.tsx](#)
 - [main.tsx](#)
 - [loader.tsx](#)
- [5. Layout Components: Header & Footer](#)
 - [AppHeader](#)
 - [AppFooter](#)
- [6. Main Page: DashboardPage](#)
- [7. Core Components](#)
 - [SearchForm](#)
 - [SelectedClinicResults](#)
 - [NearbyClinicsResults](#)
 - [ClinicCard](#)
 - [Map](#)
- [8. Data Flow & State Management](#)
 - [State Transition Flow](#)
 - [State Variables](#)
- [9. API Integration](#)
- [10. Data Caching](#)
- [11. Styling & Utilities](#)
- [12. Accessibility & Responsiveness](#)
- [13. Azure Maps](#)
- [14. Testing](#)
- [15. Main Commands](#)
- [16. React Build Output Integration with .NET Core](#)

1. Introduction

This document provides a comprehensive overview of the call-center-react-app, detailing its folder structure, main entry points, layout components, pages, and core functionality. It is intended for developers onboarding to the project or seeking a reference for architecture and component usage.

2. Technical Stack

- **Language:** TypeScript (React)
- **Framework:** React 18.x
- **UI Library:** Neutron UI SDK (HCA Healthcare Design System)
- **Mapping:** Azure Maps (via react-azure-maps, azure-maps-control)
- **Testing:** Vitest, @testing-library/react
- **Build Tool:** Vite
- **Styling:** Neutron utility classes, Bootstrap utilities, custom CSS
- **State Management:** Local React state (no Redux/Context API)

3. Project Structure Overview

```

call-center-react-app/
├── src/
│   ├── components/
│   │   ├── AppHeader/
│   │   ├── AppFooter/
│   │   ├── AppLoading/
│   │   ├── ClinicCard/
│   │   ├── Map/
│   │   ├── NearbyClinicsResults/
│   │   ├── SearchForm/
│   │   └── SelectedClinicResults/
│   ├── pages/
│   │   └── Dashboard/
│   │       └── DashboardPage.tsx
│   ├── dashboardUtils.ts
│   ├── models.ts
│   ├── main.tsx
│   ├── loader.tsx
│   ├── App.tsx
│   └── ...
├── public/
└── ...

```

- **components/:** Reusable UI and logic, grouped by feature.
- **pages/:** Main application pages (Dashboard and Error pages).
- **main.tsx:** React entry point.
- **loader.tsx:** Application rendering logic.
- **App.tsx:** Root component.

4. Entry Points: App.tsx, main.tsx & loader.tsx

App.tsx

- **Purpose:** The root React component. Sets up global layout, routing, and context providers if needed.
- **Typical Structure:**
 - Renders AppHeader , main page content (DashboardPage , ErrorPage), and AppFooter .

main.tsx

- **Purpose:** The main entry point for the React app.
- **Functionality:**

- Imports and calls `LoadNeutronApplication` from `loader.tsx`.
- Responsible for bootstrapping the app and starting the React rendering process.

loader.tsx

- **Purpose:** Contains the logic to render the React application.
- **Functionality:**
 - Imports `React`, `ReactDOM`, `React Router`, and the root `App` component.
 - Defines and exports `LoadNeutronApplication`, which renders `<App />` inside a router and strict mode, targeting the DOM node with id `root`.
 - Used by `main.tsx` to start the app.

5. Layout Components: Header & Footer

AppHeader

- **Purpose:** Displays the application title, and branding.
- **Props:** Typically none, but may accept user info or navigation links.
- **State:** Stateless or may track menu open/close.
- **Usage:** Rendered at the top of every page.

AppFooter

- **Purpose:** Shows copyright, and legal info.
- **Props:** None or minimal.
- **State:** Stateless.
- **Usage:** Rendered at the bottom of every page.

6. Main Page: DashboardPage

- **Purpose:** Central dashboard for searching clinics and certifications, showing results, and displaying nearby clinics on a map.
- **State:**
 - `selectedClinicId`, `selectedCertificationId`: Track current selections
 - `dateSelected`: 'today' or 'tomorrow' for service hours
 - `certifications`, `clinics`: Data loaded from API
 - `dashboardData`, `selectedClinicData`, `nearbyClinicsData`: Dashboard and filtered results
 - `showSelectedClinic`, `showNearbyOptions`, `showMap`: UI state for conditional rendering
 - `dashboardLoading`, `dashboardError`: Loading and error state
 - `dashboardDataCache`: In-memory cache for dashboard data
- **Props Passed:**
 - To `SearchForm`: `certifications`, `clinics`, `selectedClinicId`, `selectedCertificationId`, `onSearch`, `onSelectionChange`
 - To `SelectedClinicResults`: `certificationName`, `clinicName`, `selectedClinicHours`, `dateSelected`, `setDateSelected`, `showNearby`, `setShowNearby`

- To `NearbyClinicsResults`: `nearbyClinicsData`, `certificationName`, `dateSelected`, `showMap`, `setShowMap`, `getClinicHours`, `mapCenter`, `markers`
- **Effects:**
 - Fetches certifications and clinics from API endpoints.
 - Filters, maps, and sorts data for display.
 - Manages conditional rendering for search, results, and map.
 - Handles in-memory caching and error/loading UI states.

7. Core Components

SearchForm

- **Purpose:** Lets users select a clinic and certification/service to search.
- **Props:** `certifications`, `clinics`, `selectedClinicId`, `selectedCertificationId`, `onSearch`, `onSelectionChange`
- **Stateless.**
- **Behavior:** Searchable clinic input (min 3 chars, highlights matches), certification dropdown, disables Search button until both are selected, calls `onSearch` on submit, calls `onSelectionChange` on selection changes.

SelectedClinicResults

- **Purpose:** Shows service hours for the selected clinic and certification.
- **Props:** `certificationName`, `clinicName`, `selectedClinicHours`, `dateSelected`, `setDateSelected`, `showNearby`, `setShowNearby`
- **Stateless.**
- **Behavior:** Displays available hours or a message if none; allows toggling between today/tomorrow; button to show/hide nearby clinics.

NearbyClinicsResults

- **Purpose:** Shows nearby clinics as a list or on a map.
- **Props:** `nearbyClinicsData`, `certificationName`, `dateSelected`, `showMap`, `setShowMap`, `getClinicHours`, `mapCenter`, `markers` (optional)
- **Stateless.**
- **Behavior:** Toggle between list and Azure Maps view; shows `ClinicCard` for each clinic or map markers; handles empty state.

ClinicCard

- **Purpose:** Displays details for a single clinic.
- **Props:**
 - `name`, `serviceHours`, `dateSelected`, `address`, `location`, `phoneNumber`, `distance`
- **State:** Stateless.
- **Functionality:**
 - Shows clinic name, address, phone, and service hours.
 - Used in both selected and nearby clinics results.

Map

- **Purpose:** Renders Azure Maps with clinic markers.
- **Props:**
 - `options` : Map options (center, zoom, authentication).
 - `markers` : Array of marker data (position, name, hours, etc.).
 - `dateSelected` , `controls` , `styleOptions`
- **State:** Stateless.
- **Functionality:**
 - Displays clinics on a map.
 - Handles marker selection and popups.

8. Data Flow & State Management

- **State is managed in DashboardPage** and passed down as props.
- **API calls** are made in DashboardPage for clinics and certifications.
- **Conditional rendering** is used for search, results, and map views.
- **Component communication** is via props and callback functions.

Below is a diagram showing how the main components in `DashboardPage` are shown or hidden based on user actions and state variables.

Initial State	After Search	Show Nearby Options
<code>!showSelectedClinic</code>	<code>showSelectedClinic</code> && <code>!showNearbyOptions</code>	<code>showSelectedClinic</code> && <code>showNearbyOptions</code>
SearchForm (centered)	SearchForm (centered) SelectedClinicResults (centered)	SearchForm (left column) SelectedClinicResults (left column) NearbyClinicsResults (right column)

State Transition Flow

Initial State

- Only `SearchForm` is visible.
- User selects clinic/service and clicks search.
- → `showSelectedClinic` becomes `true` .

After Search

- `SearchForm` and `SelectedClinicResults` are visible.
- User clicks "Show Nearby Options" button.
- → `showNearbyOptions` becomes `true` .

Show Nearby Options

- `SearchForm` and `SelectedClinicResults` are shown in the left column.
- `NearbyClinicsResults` is shown in the right column.
- If user changes clinic/service, `showNearbyOptions` is set to `false` (returns to previous state).

State Variables

- `showSelectedClinic` : Controls if results are shown after a search.
- `showNearbyOptions` : Controls if nearby clinics are shown.

9. API Integration

- **Endpoints:**
 - `/api/certifications` : Returns available certifications/services.
 - `/api/clinics` : Returns available clinics.
 - `/api/servicesdashboard?certId={certificationId}&clinicId={clinicId}&date={YYYY-MM-DD}` : Fetches dashboard data for the selected clinic, certification, and date. Returns an array of clinics with service hours, staff, and coordinates.
- **Data Mapping:**
 - Certifications are filtered and renamed for display.
 - Clinics are mapped to include display name (`name (city, state)`).

10. Data Caching

- **Files:** `src/dashboardUtils.ts` , `src/pages/Dashboard/DashboardPage.tsx`
- **How it works:**

In-memory caching of dashboard data for each unique clinic/certification/date combination using a cache object and expiration time (see `getOrFetchDashboardData`).
- **Caching Strategy:**
 - Dashboard data is cached in memory per selected clinic, certification, and date for the session only (not persisted).
 - The cache for each clinic/certification/date combination is refreshed every 5 minutes (expiration time is 5 minutes).
 - Cache is invalidated on refresh, explicit reload, or when a new clinic/certification/date is selected.

11. Styling & Utilities

- **Neutron Design System:** Uses Neutron React components and utility classes for consistent UI.
- **CSS:** Component-specific CSS files for custom styles.
- **Bootstrap Utilities:** For spacing, flexbox, and layout.

12. Accessibility & Responsiveness

- **Accessibility:**

- The UI leverages Neutron components, which are built to meet HCA accessibility standards (WCAG 2.1 AA).
- Semantic HTML and ARIA attributes are used where appropriate.
- All interactive elements (buttons, forms, map controls) are keyboard accessible.
- Color contrast and focus indicators are handled by the design system.
- **Responsiveness:**
 - The layout is fully responsive, adapting to desktop, tablet, and mobile screens.
 - Neutron and Bootstrap utility classes are used for grid, spacing, and flexbox layouts.
 - The map and results panels stack vertically on small screens for usability.

13. Azure Maps

- **Integration:** Uses Azure Maps for displaying clinic locations on a map (in NearbyClinicsResults and Map components).
- **Libraries:** `react-azure-maps`, `azure-maps-control`
- **Authentication:**
 - Uses Azure Maps client ID and a token fetched from `/api/azure-maps-token`.
 - Token is cached and reused until near expiration for performance.
- **Usage:**
 - Map view is available for nearby clinics if valid coordinates are present.
 - Markers are built for each clinic with valid coordinates.
 - Map controls and style options are configured for usability and HCA branding.

14. Testing

- **Test Framework:** Vitest, `@testing-library/react`
- **Test Location:** `src/components/*/*.test.tsx` and similar for page-level tests
- **How to run tests:**
 - `npm test` OR `npm run test` — Runs all tests in watch mode
- **Test Patterns:**
 - Component tests use React Testing Library for rendering and user interaction
 - Mocks are used for API calls and external dependencies

15. Main Commands

- `npm install` — Install all dependencies
- `npm run dev` — Start the development server (Vite, hot reload)
- `npm run build` — Build the production bundle
- `npm run preview` — Preview the production build locally
- `npm run lint` — Run ESLint on the codebase
- `npm run test` — Run all tests in watch mode

16. React Build Output Integration with .NET Core

The build output of the React app is directed to the [ASP.NET](#) Core app's `wwwroot` folder by setting the `outDir` in `call-center-react-app/vite.config.ts`.

Running `npm run build` in the React app places the static files directly into the [ASP.NET](#) Core app's `wwwroot` folder, allowing the .NET Core backend to serve the React frontend seamlessly.